

RouteCard 仕様書

時刻付きルート共有アプリ（仮称: RouteCard）

- 版数: v1.0（初版）
- 作成日: 2026-07-06
- 対象: 個人開発（開発者本人が主ユーザー）

1. 背景と目的

1.1 解決したい課題

Googleマップの「ルートを共有」機能で生成されるURLには、**出発時刻・選択した電車（便）の情報**が含まれない。そのため「明日9:12発のこの電車で行こう」という情報を他人に伝える手段がスクリーンショットしかない。

1.2 目的

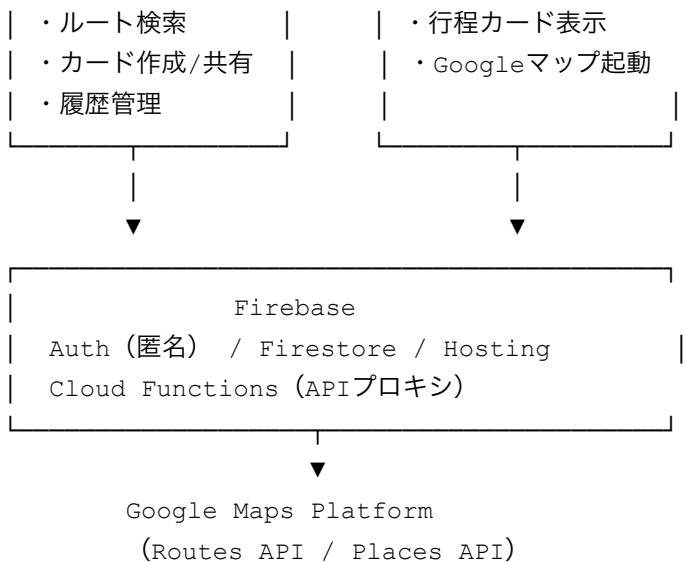
- 出発地・目的地・日時を指定した乗換ルートを検索し、**時刻付きの行程カード**として保存・共有できるようにする
- 共有相手は**アプリ不要**（URLをブラウザで開くだけ）で行程を確認でき、ワンタップでGoogleマップを開ける

1.3 前提となる制約（重要）

- Googleマップの公式URL仕様には出発時刻を渡すパラメータが存在しない。したがって**Googleマップアプリを時刻付きで開かせることは不可能**
- 設計方針: 時刻情報は**RouteCard側が保持**し、地図表示・ナビは**Googleマップに任せる**

2. 全体構成





- **Expoアプリ**: ルートの検索・カード作成・共有・履歴管理を行う。利用者は開発者本人のみを想定
- **共有ページ**: Firebase Hostingでホストする静的Webページ。`/r/{cardId}` 形式のURLでカードを表示
- **Cloud Functions**: Routes API / Places APIの呼び出しを仲介し、APIキーをクライアントに露出させない

3. 機能仕様（Expoアプリ）

3.1 画面一覧

#	画面名	役割
S1	ルート作成	出発地・目的地・日時の入力と検索
S2	ルート選択	検索結果の候補一覧と詳細確認
S3	カード編集・共有	メモ追加、カード生成、共有シート起動
S4	履歴	作成済みカードの一覧・再共有・複製・削除

3.2 S1: ルート作成画面

入力項目

項目	仕様
----	----

出発地	テキスト入力。Places Autocomplete（Cloud Functions経由）で駅名・施設名を補完。 placeIdと表示名を保持
目的地	同上
日時	日付＋時刻ピッカー。デフォルトは「現在時刻＋10分」
時刻の基準	「出発」 / 「到着」 のトグル。到着指定なら arrivalTime として検索
移動手段	v1では公共交通機関（TRANSIT）固定。UI上は将来拡張を見越してセグメント表示のみ

動作

- 「検索」 タップで Cloud Functions 経由の Routes API 検索を実行
- 通信中はローディング表示。失敗時はエラーメッセージ＋再試行ボタン
- 出発地・目的地の入れ替えボタン（↕）を設置

3.3 S2: ルート選択画面

一覧表示（1候補あたり）

09:12発 → 09:48着（36分）
東海道線 ほか / 乗換1回 / ￥580

- 最大5候補程度を表示。出発時刻の早い順
- 各候補をタップすると詳細を展開:
 - 区間ごとの「乗車駅・発時刻 → 路線名（行先） → 降車駅・着時刻」
 - 徒歩区間は「徒歩 ○分」表示
- 「このルートでカードを作る」 ボタンでS3へ

3.4 S3: カード編集・共有画面

- 選択したルートのプレビュー（共有ページと同じ見た目）
- メモ入力（例: 「北改札前に9:00集合」。最大200文字）
- 「共有する」 タップで以下を実行:

1. Firestoreにカードを保存（ `routeCards/{cardId}` ）

2. 共有URL `https://<hosting-domain>/r/{cardId}` を生成

3. OS共有シート (React Native `Share` API) を起動し、テキスト+URLを渡す

共有テキストのフォーマット例

【7/7(火) 横浜 → 品川】
09:12発 → 09:48着 (東海道線・乗換1回)
メモ: 北改札前に9:00集合
`https://routecard.example.com/r/abc123`

3.5 S4: 履歴画面

- 作成済みカードを新しい順に一覧表示 (日付・区間・出発時刻)
 - 各カードの操作:
 - 再共有: 共有シートを再起動
 - 複製: 同じ区間で日時だけ変えて再検索 (S1に値を引き継いで遷移)
 - 削除: Firestoreから削除 (確認ダイアログあり)
 - 過去日付のカードはグレースアウト表示
-

4. 機能仕様 (共有ページ)

4.1 表示内容

URL `/r/{cardId}` にアクセスすると、Firestoreからカードを読み込んで表示する。

1. ヘッダー: 日付・「横浜 → 品川」・所要時間
2. タイムライン: 区間ごとの発着時刻・駅名・路線名を縦のタイムラインで表示 (乗換アプリ風)
3. メモ: 作成者のメモ (あれば)
4. アクション: 「Googleマップで開く」ボタン
 - リンク先: `https://www.google.com/maps/dir/?api=1&origin=<出発地>&destination=<目的地>&travelmode=transit`
 - ボタン付近に注記: 「時刻はこのカードを参照してください (Googleマップには引き継がれません)」
5. フッター注記: 「〇月〇日時点の検索結果です。最新の運行情報はご確認ください」

4.2 OGP対応

LINEで共有されることが多いため、OGPメタタグを動的に出力する（Cloud Functions によるSSR、またはprerender）。

- `og:title`: 「7/7 09:12 横浜 → 品川」
- `og:description`: 「東海道線・乗換1回・09:48着」

4.3 エラー・期限切れ

- 存在しないcardId: 「カードが見つかりません」表示
- `expiresAt` 超過で削除済みの場合も同様

5. データ仕様

5.1 Firestore スキーマ

```
routeCards/{cardId}
├ origin:      { name: string, placeId: string, lat: number, lng: number }
├ destination: { name: string, placeId: string, lat: number, lng: number }
├ searchType:  "departure" | "arrival"
├ departureTime: Timestamp           // 行程の出発時刻
├ arrivalTime:  Timestamp           // 行程の到着時刻
├ legs: [      // 区間の配列
|   {
|     mode:      "TRANSIT" | "WALK",
|     lineName:  string | null,    // 例: "東海道線" (徒歩はnull)
|     headsign:  string | null,    // 行先表示
|     depStop:   string,           // 乗車駅名
|     depTime:   Timestamp,
|     arrStop:   string,           // 降車駅名
|     arrTime:   Timestamp,
|     durationMinutes: number
|   }
├ ]
├ fare:        number | null      // 円。取得できない場合null
├ memo:        string             // 最大200文字
├ createdBy:   string             // 匿名認証UID
├ createdAt:   Timestamp
├ expiresAt:   Timestamp          // 行程日の7日後 (TTL削除対象)
```

- cardIdは推測困難なランダムID（Firestore自動ID）とし、URLを知る人だけがアクセスできる形

にする

5.2 セキュリティルール（方針）

```
match /routeCards/{cardId} {  
  allow read:    if true;                                // URLを知っていれば閲覧可  
  allow create:  if request.auth != null;                 // 匿名認証済みのみ  
  allow update, delete:  
    if request.auth.uid == resource.data.createdBy;  
}
```

- FirestoreのTTLポリシーで `expiresAt` による自動削除を設定
-

6. 外部API仕様

6.1 Routes API（computeRoutes）

- エンドポイント: `routes.googleapis.com/directions/v2:computeRoutes`
- 主要パラメータ:
 - `travelMode: "TRANSIT"`
 - `departureTime` または `arrivalTime`（RFC 3339形式）
 - `computeAlternativeRoutes: true`（複数候補取得）
- FieldMaskで取得フィールドを絞る（`legs.steps.transitDetails` など必要分のみ）→ レスポンス削減とコスト最適化
- 呼び出しは必ずCloud Functions経由（APIキー秘匿）

6.2 Places API（Autocomplete）

- 駅名・施設名の補完に使用。 `components=country:jp` で日本に限定
- セッショントークンを利用してコストを抑える
- こちらもCloud Functions経由

6.3 コスト管理

- Google Cloud で予算アラートを設定（例: 月500円で通知）
- APIキーにはHTTPリファラ/IP制限とAPI制限（Routes/Placesのみ許可）を設定

7. 技術スタック

レイヤ	技術
アプリ	Expo (React Native) + expo-router
共有機能	React Native <code>Share</code> API
認証	Firebase Auth (匿名認証)
DB	Cloud Firestore
共有ページ	Firebase Hosting (+ Cloud FunctionsでOGP用SSR)
APIプロキシ	Cloud Functions (Node.js)
地図データ	Google Maps Platform: Routes API / Places API
開発環境	Windows + VSCode + Claude Code

8. 非機能要件・注意事項

- **データの正確性:** Routes APIの乗換データは乗換専用アプリよりダイヤ・運賃の精度が粗い場合がある。カードには検索日を明示し、確定情報として扱わない
- **オフライン:** アプリ側は履歴のローカルキャッシュ表示のみ対応。検索はオンライン必須
- **プライバシー:** カードには位置履歴が含まれるため、URL非公開前提+TTL自動削除で対応。共有ページに「このカードは〇日後に自動削除されます」と表示
- **多言語:** v1は日本語のみ

9. 開発ステップ（推奨順）

1. **API疎通:** Cloud FunctionsからRoutes API (TRANSIT + departureTime) を叩き、行程JSONが取れることを確認
2. **共有ページ:** 手動で作ったFirestoreデータを `/r/{cardId}` で表示できるところまで
3. **アプリUI:** S1→S2→S3の順で実装。共有シート連携まで

4. 履歴・複製・TTL: S4と運用まわり
 5. 仕上げ: OGP、エラーハンドリング、予算アラート
-

10. 将来拡張（v2以降の候補）

- 車・徒歩モード対応（時刻の意味が薄いため優先度低）
- カードの画像書き出し（LINEに画像としても貼れる）
- 複数ルート候補を1枚のカードにまとめて「どれで行く？」と相手に選ばせる機能
- リマインダー連携（出発30分前に通知）